

Практичне завдання

Валідація моделей: code competition

Мета: навчитися будувати коректну стратегію валідації, яка чесно передбачає якість моделі на нових даних. Ви побачите на практиці, чому «наївна» валідація (звичайний випадковий поділ) може суттєво вводити в оману.

Це **code competition**: ви не здаєте ноутбук із графіками. Ви здаєте один Python-файл із двома функціями `train_model()` та `predict()`. Ваш код запускається на прихованому тестовому наборі, якого ви ніколи не бачите.

Частина 1. Оберіть датасет

- Оберіть один датасет із таблиці нижче — будь-який на ваш смак.
- Для обраного варіанту вам відомо: тип задачі, метрику оцінювання та назву цільового стовпчика.
- Вам надаються дві частини даних (обидві з таргетом): `ds_N_train.csv` — тренувальні дані, та `ds_N_val.csv` — валідаційні дані (300 рядків).
- Прихований тестовий набір залишається в інструктора й використовується для фінального оцінювання.

Частина 2. Побудова моделі та валідації

- Навчіть модель на `ds_N_train.csv` так, щоб вона якнайкраще працювала на прихованих даних.
- Оцініть якість моделі звичайним випадковим K-fold на тренувальних даних.
- Оцініть ту саму модель на `ds_N_val.csv`. Порівняйте два отримані значення метрики.
- Якщо вони суттєво розходяться — ваша валідаційна стратегія неправильна, і на прихованому тесті ви отримаєте радше значення, ближче до `ds_N_val.csv`, ніж до оптимістичного K-fold.

- e. Підберіть таку стратегію локальної валідації, щоб ваша оцінка на train максимально збігалася з оцінкою на `ds_N_val.csv`. Варто спробувати: `GroupKFold`, `TimeSeriesSplit`, `adversarial validation`, стратифікацію — залежно від того, що ви помітите в даних.

Частина 3. Оформлення рішення (контракт)

- a. Візьміть файл-приклад `solution_example.py` як шаблон.
- b. `train_model(train_df)` — отримує тренувальні дані з таргетом. Робить увесь препроцесинг, навчання, підбір гіперпараметрів. Повертає об'єкт `state` (модель / словник / кортеж — будь-що).
- c. `predict(state, eval_df)` — отримує `state` і набір без таргету. Повертає 1-D `np.ndarray` довжини `len(eval_df)` у тому самому порядку рядків.
- d. Зверху файлу вкажіть `DS_ID`, `TARGET`, `TASK` під ваш обраний датасет.

Частина 4. Правила

- a. Для класифікації `predict()` повертає ймовірність позитивного класу (потрібно для ROC-AUC), а не 0/1.
- b. Не читайте файли всередині `train_model()` / `predict()` — усі дані приходять через аргументи.
- c. Використовуйте бібліотеки, доступні в Google Colab без додаткового встановлення.

Частина 5. Метрика та оцінювання

- a. Регресія: R^2 (більше — краще; 1 — ідеал, 0 — рівень середнього, менше 0 — гірше за середнє).
- b. Класифікація: ROC-AUC (більше — краще).
- c. Інструктор запустить ваші `train_model()` / `predict()` на прихованому тестовому наборі й порахує метрику.
- d. Порівнюватимуться дві величини: метрика на вашому `ds_N_val.csv` (ваша власна оцінка) і метрика на прихованому тесті (реальна якість).
- e. Оцінюється не лише абсолютна якість моделі, а й наскільки близько ваша валідаційна оцінка збіглася з реальною. Велике розходження означає, що ваша стратегія валідації не відображає реальність — навіть якщо сама модель непогана.

Оберіть датасет із таблиці нижче — будь-який на ваш смак.

Датасет	Тип задачі	Метрика	Цільовий стовпчик	Коментар
ds_1	Регресія	R^2	total_UPDRS	Тяжкість симптомів за голосовими вимірами.
ds_2	Регресія	R^2	sales	Продажі товару в магазині (часовий ряд).
ds_3	Регресія	R^2	Sales	Продажі мережі магазинів (часовий ряд).
ds_4	Класифікація	ROC-AUC	SeriousDlqin2yrs	Ймовірність фінансового дефолту клієнта.
ds_5	Класифікація	ROC-AUC	target	Ймовірність страхового випадку.

Питання для самоконтролю

- Чому не можна оцінювати модель на тих самих даних, на яких вона навчалась?
- Чим валідаційна вибірка відрізняється від тестової?
- Коли звичайний випадковий K-fold дає надто оптимістичну оцінку якості?
- Що таке GroupKFold і TimeSeriesSplit, і в яких ситуаціях вони потрібні?
- Що показує adversarial validation?
- Чому ваша оцінка на валідації могла розійтися з результатом на прихованому тесті — і як це виправити?

Що потрібно здати?

Після завершення практичної роботи:

1. Один .py файл, названий <прізвище>_ds<N>.py.
2. Файл має самостійно містити всі імпорти й проходити локальну самоперевірку.
3. Завантажте файл через форму здачі домашнього завдання.